

Ethical Hacking Basics

HACKING

THE ART OF EXPLOITATION

with emphasis on Web Applications Security Related Issues

Topics To Be Covered

Ethical Hacking - ?

Why – Ethical Hacking ?

Ethical Hacking - Process

Web Apps Security- Possible Flaws

Web Apps Security- Known Attacks

What is Ethical Hacking

Also Called – Attack & Penetration Testing, White-hat hacking, Red teaming

Hacking

Process of breaking into systems for:

- Personal or Commercial Gains
- Malicious Intent – Causing sever damage to Information & Assets

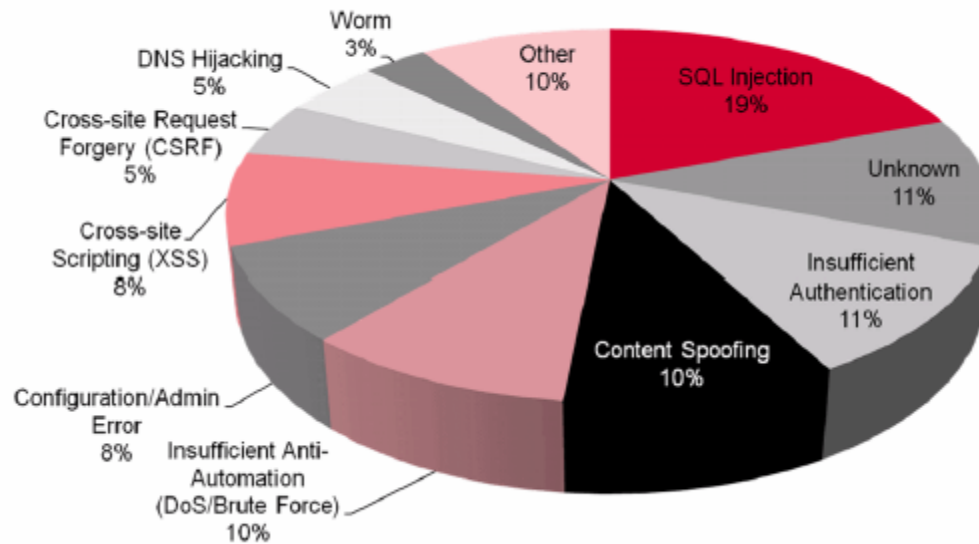
Ethical

Conforming to accepted professional standards of conduct

Black-hat – Bad guys

White-hat - Good Guys

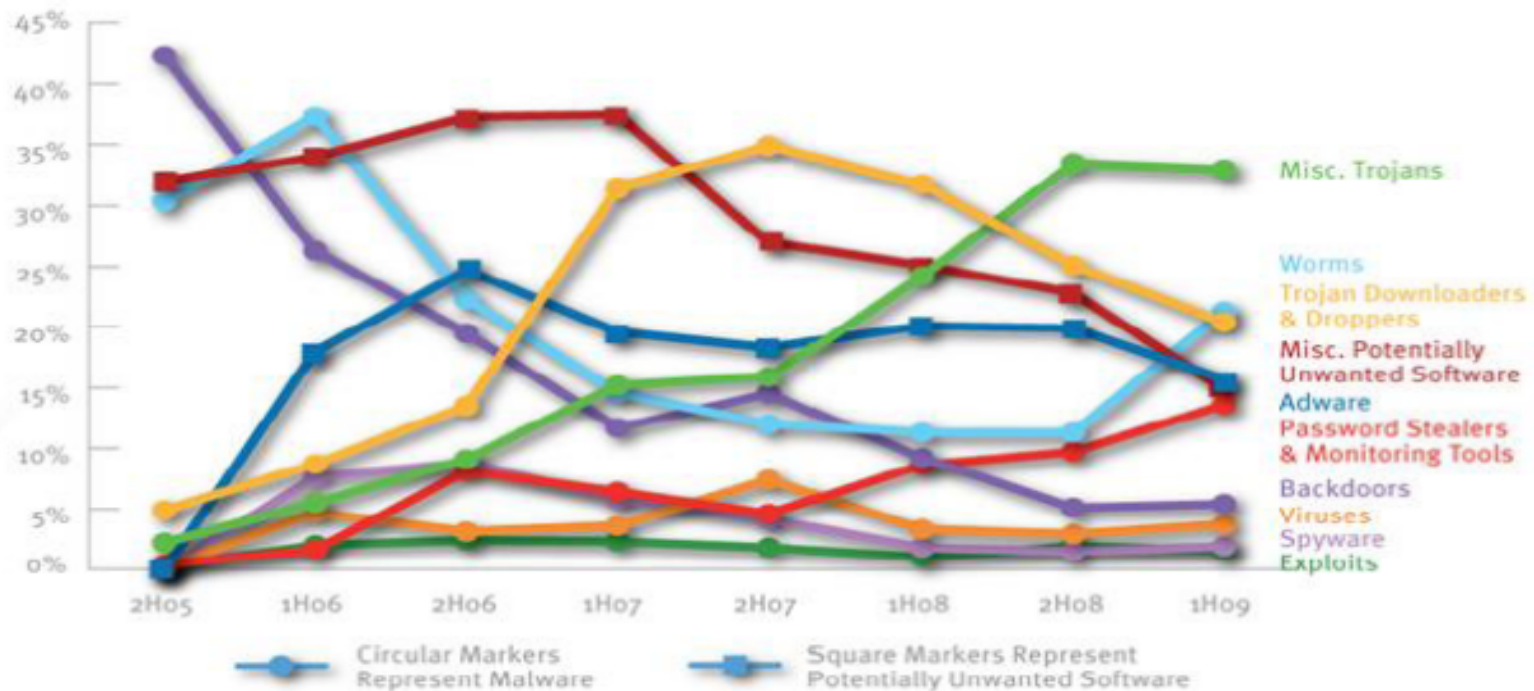
Why – Ethical Hacking



**THE WEB HACKING INCIDENTS DATABASE 2009 BI-ANNUAL REPORT AUGUST 2009
BREACH SECURITY CORPORATION USA**

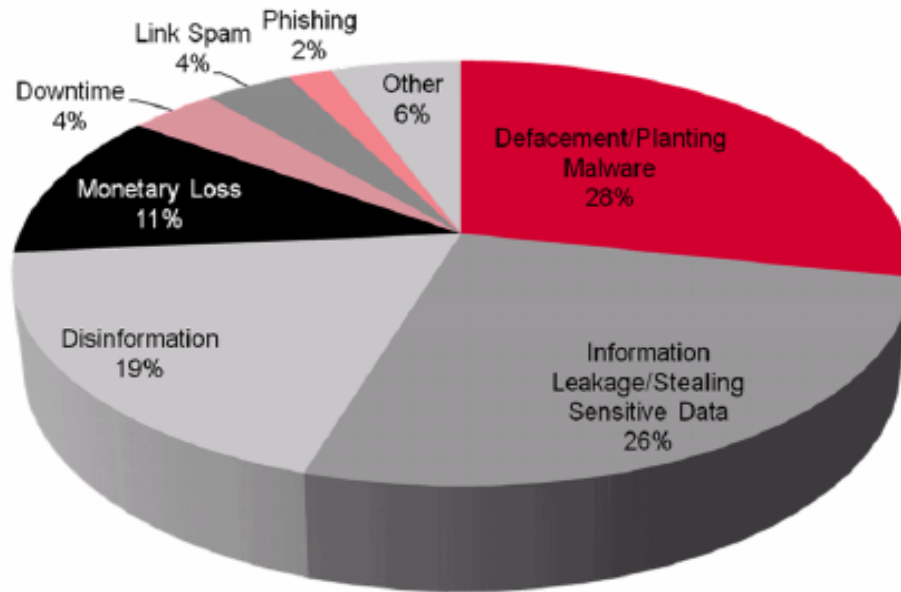


Why – Ethical Hacking



Trends of different types of attacks(2005-2009)

Why – Ethical Hacking



**THE WEB HACKING INCIDENTS DATABASE 2009 BI-ANNUAL REPORT AUGUST 2009
BREACH SECURITY CORPORATION USA**

Idea Behind Ethical Hacking

- As its name implies, penetration testing is a series of activities undertaken to identify and exploit security vulnerabilities.
- The idea is to find out how easy or difficult it might be for someone to “penetrate” an organization’s security controls or to gain unauthorized access to its information and information systems.

Penetration Testing Strategies

➤ External vs. internal testing:

- ▣ External testing refers to attacks on the organization's network perimeters using procedures performed from outside the Internet or Extranet by targeting Domain Name Server (DNS), email server, web server or firewall.
- ▣ Internal testing is performed from within the organization's technology environment. The focus is to see what an authorized user could do to penetrate specific information resources within the organization's network.

➤ Blind Vs. Double blind vs. targeted testing:

- ▣ The penetration testing team must use publicly available information (such as company web-site and domain name registry, Internet discussion board) to gather information about the target and conduct its penetration tests. Target organization's IT staff is already aware.
- ▣ Double-blind testing extends the blind testing strategy in that the organization's IT and security staff are not notified or informed beforehand and are "blind" to the planned testing activities.
- ▣ Targeted testing (often referred to as the "lights-turned-on" approach) in this testing team is aware of the testing activities and is provided information concerning the target and the network design.

Identification of Vulnerabilities

Look for

- ❑ Un patched / Possible Vulnerabilities in OS and other software
- ❑ Open Ports
- ❑ Weak Passwords
- ❑ Weak Configurations
- ❑ Insecure Programming
- ❑ Logic Flaws
- ❑ Weak Access Control

You need hands on Tools Like:

Vulnerability Scanners : Nessus, ISS, SARA, SAINT, PAROS

Listening to Traffic : Ethercap, tcpdump, wireshark

Password Crackers : John the ripper, LC4, Pwdump, cain and abel

Intercepting Web Traffic: Achilles, Whisker, Legion

Scanning Ports: Nmap , Superscan

Finding the OS: P0f v2

web server scanner: Nikto

Other useful tools: PING, whois, Traceroute, DIG, nslookup, sam spade, FScan, Hping, Firewalk, netcat, tcpdump, ssh, telnet, SNMP Scanner

Attack – Exploit the vulnerabilities

- **Network Infrastructure Attacks**
- **Operating System Attacks**
- **Application Specific Attacks**
(Will be discussed in detail in later slides)



First Step:

Understand Application Environment

The header and any page returned from a HEAD or OPTIONS request will usually contain a SERVER: string or similar detailing the Web server software version and possibly the scripting environment or operating system in use.

```
yourname@S214-01u:~$ nc www.ccsf.edu 80
HEAD / HTTP/1.0

HTTP/1.1 200 OK
Date: Sat, 10 Feb 2007 03:47:43 GMT
Server: Apache/1.3.26 (Unix) mod_ssl/2.8.10 OpenSSL/0.9.7d
Connection: close
Content-Type: text/html; charset=iso-8859-1
```

Contd..

Understanding Application Environment

- **Manipulate inputs** in order to elicit a scripting error
In the example below the most obvious variable (ItemID) has been manipulated to fingerprint the Web application environment



■ **TCP/ICMP** and **Service Finger Printing**

Done using traditional fingerprinting tools such as **Nmap** and **Queso**, or the more recent application fingerprinting tools **Amap** and **WebServerFP**, the penetration tester can gain a more accurate idea of the underlying operating systems and Web application environment than through many other methods. NMAP and Queso examine the nature of the host's TCP/IP implementation to determine the **operating system** and, in some cases, the **kernel version** and **patch level**. Application fingerprinting tools rely on data such as Server HTTP headers to identify the host's application software.

Contd..Understanding Application Environment

- Investigate the format and wording of 404/other error pages .
Will often give away the software versions of the scripting language in use.
- Test for recognised file types/extensions/directories
Many Web services (such as Microsoft IIS) will react differently to a request for a known and supported file extension than an unknown extension. The tester should attempt to request common file extensions such as .ASP, .HTM, .PHP, .EXE and watch for any unusual output or error codes.

GET /blah.idq HTTP/1.0

HTTP/1.1 200 OK

Server: Microsoft-IIS/5.0

Date: Wed, 04 Jun 2003 11:12:24 GMT

Content-Type: text/html

<HTML>The IDQ file blah.idq could not be found.

- Examine source of available pages
The source code from the immediately accessible pages of the application front-end may give clues as to the underlying application environment.

<title>Home Page</title>

<meta content="Microsoft Visual Studio 7.0" name="GENERATOR">

<meta content="C#" name="CODE_LANGUAGE">

<meta content="JavaScript" name="vs_defaultClientScript">

In this situation, the developer appears to be using MS Visual Studio 7. The underlying environment is likely to be Microsoft IIS 5.0 with .NET framework.

Known Problems with Web Applications

- ❑ Links and pointers to executables with improper file permissions
- ❑ Use of Hidden form elements values
- ❑ JavaScript and Client side code
- ❑ Loose Authentication Mechanism
- ❑ Poor Input Validations on forms submission

Known Attacks

- ❑ SQL Injection Attack
- ❑ Code and Content Injection Attack
- ❑ Cross site Scripting Attack
- ❑ Path Traversal and URLs
- ❑ Buffer Overflows
- ❑ Exploit Session Security
- ❑ Capture Logic Flaws
- ❑ Cookies
- ❑ Webbugs

SQL Injection Attacks Example

- This is one of the most commonly used examples of an SQL injection vulnerability, as it is easy to understand for non-SQL-developers and highlights the extent and severity of these vulnerabilities. One of the simplest ways to validate a user on a Web site is by providing them with a form, which prompts for a username and password. When the form is submitted to the login script (eg. login.asp), the username and password fields are used as variables within an SQL query.

Examine the following code (using MS Access DB as our backend):

```
user = Request.form("user") pass = Request.form("pass") Set Conn =  
Server.CreateObject("ADODB.Connection") Set Rs = Server.CreateObject("ADODB.Recordset")  
Conn.Open (dsn) SQL = "SELECT C=COUNT(*) FROM users where pass=" & pass & " and user=" &  
& user & "" rs.open (sql,conn) if rs.eof or rs.bof then response.write "Database Error" else if rs("C")  
< 1 then response.write "Invalid Credentials" else response.write "Logged In" end if end if
```

In this scenario, no sanity or validity checking is being performed on the user and pass variables from our form inputs. The developer may have client-side (eg. Javascript) checks on the inputs, but as has been demonstrated in the first part of this series, any attacker who understands HTML can bypass these restrictions. If the attacker were to submit the following credentials to our login script:

```
user: test' OR '1'='1  
pass: test
```

the resulting SQL query would look as follows:

```
SELECT * FROM users where pass='test' and user='test' OR '1' = '1'
```

Code and Content Injection

Example: A simple message board in PHP

- The following snippet of PHP code is used to display posts for a particular message board. It retrieves the messageid GET variable from the user and opens a file \$messageid.txt under /var/www/forum:
- ```
<?php include('/var/www/template/header.inc'); if (isset($_GET['messageid']) && file_exists('/var/www/forum/' . stripslashes($messageid) . '.txt') && is_numeric($messageid)) { include('/var/www/forum/' . stripslashes($messageid) . '.txt'); } else { include('/var/www/template/error.inc'); } include('/var/www/template/footer.inc'); ?>
```
- Although the is\_numeric() test prevents the user from entering a file path as the messageid, the content of the message file is not checked in any way. (The problem with allowing unchecked entry of file paths is explained later) If the message contained PHP code, it would be included and therefore executed by the server.
- A simple method of exploiting this example vulnerability would be to post to the message board a simple chunk of code in the language of the application (PHP in this example), then view the post and see if the output indicates the code has been executed.

# Cross Site Scripting Attack

---

- A simple example of XSS can be seen in the following URL:
- `http://server.example.com/browse.cfm?categoryID=1&name=Books`
- In this example the content of the 'name' parameter is displayed on the returned page. A user could submit the following request:
- A good example would be as follows:
- `http://server.example.com/browse.cfm?categoryID=1&name=<script>alert(document.cookie);</script>`
- In this case, we have managed to inject Java script into the resulting page. The relevant cookie (if any) for this session would be displayed in a popup box upon submitting this request.
- This can be abused in a number of ways, depending on the intentions of the attacker. A short piece of Java script to submit a user's cookie to an arbitrary site could be placed into this URL. The request could then be hex-encoded and sent to another user, in the hope that they open the URL.
- Upon clicking the trusted link, the user's cookie would be submitted to the external site. If the original site relies on cookies alone for authentication, the user's account would be compromised.

# Path Traversal and URLs

---

```
<?php include('/var/www/template/header.inc');
if (isset($_GET['file'])
 { $fp = fopen("$file" . ".html", "r"); }
else { $fp = fopen("main.html", "r"); }
 include('/var/www/template/footer.inc'); ?>
Look at The fopen("$file" , "r");
```

- ...the attacker would be able to traverse to any file readable by the Web application.
- <http://www.example.com/index.php?file=../../../../../etc/passwd>
- This request would return the contents of /etc/passwd unless additional stripping of the path character (/.) had been performed on the file variable.  
lo
- This problem is compounded by the automatic handling of URIs by many modern Web scripting technologies, including PHP, Java and Microsoft's .NET. If this is supported on the target environment, vulnerable applications can be used as an open relay or proxy:
- <http://www.example.com/index.php?file=http://www.google.com/>
- This flaw is one of the easiest security issues to spot and rectify, although it remains common on smaller sites whose application code performs basic content wrapping. The problem can be mitigated in two ways. First, by implementing an internal numeric index to the documents or, as in our message board code, using files named in numeric sequence with a static prefix and suffix. Second, by stripping any path characters such as [/\.] which attackers could use to access resources outside of the application's directory tree.

# Cookies

---

- ❑ Cookies are a mechanism for maintaining persistent data on the client side of a HTTP exchange.
- ❑ Take the following example of an exchange of cookies. The client requests a resource, and receives in the headers of the response:
- ❑ Set-Cookie: PASSWORD=g0d; path=/; expires=Friday, 20-Jul-03 23:23:23 GMT
- ❑ When the client requests a resource in path "/" on this server, it sends:
- ❑ Cookie: PASSWORD=g0d
- ❑ Cookies are often used to authenticate users to an application. If the user's cookie is stolen or captured, an attacker can impersonate that user. There have been numerous browser vulnerabilities in the past that allow attackers to steal known cookies

# WEB BUGS

---

- ❑ 1-pixel x 1-pixel image file (usually transparent)
  - ❑ Referenced in an <IMG> tag
  - ❑ Usually works with a cookie
  - ❑ Purpose similar to that of spyware and adware
  - ❑ Comes from third-party companies specializing in data collection

# Session Security and session-IDs

---

- Session-ID :A unique string given to a user after logging in
- Passed via GET method in url (PHP) or stored in the form of cookies.
- PHP automatically modifies all links at runtime to add the PHPSESSID to the link URL, thereby persisting state. Not only is this vulnerable to replay attacks (since the Session ID forms part of the URL), it trivializes it -- searching proxy logs, viewing browser histories or social engineering a user to paste you a URL as they see it (containing their Session ID) are all common methods of attack.

# Buffer Over Flows

---

- ❑ Web applications developed in a language that employs static buffers (such as C/C++) may be vulnerable to traditional binary attacks.
- ❑ A buffer overflow occurs when a program attempts to store more data in a static buffer than intended. The additional data overwrites and corrupts adjacent blocks of memory, and can allow an attacker to take control of the flow of execution and inject arbitrary instructions.



# Logic Flaws

- Logic flaws are a broad category of vulnerability encompassing most bugs which do not explicitly fall into another category. A logic flaw is a failure in the web application's logic to correctly perform conditional branching or apply security. For example, take the following snippet of PHP code:

```
<?php
$a=false; $b=true; $c=false;

if ($b && $c || $a && $c || $b)
 echo "True";
else
 echo "False";
?>
```

- The above code is attempting to ensure that two out of the three variables are set before returning true. The logic flaw exists in that, given the operator precedence present in PHP, simply having \$b equal to true will cause the if statement to succeed. This can be patched by replacing the if statement with the following:

```
if ($b && ($c || $a) && ($c || $b))
```

# Hidden Input Elements

---

There have been numerous examples of poorly written ordering systems in past that would allow users to save a local copy of order confirmation pages, edit HIDDEN variables such as price and delivery costs, and resubmit their request.

The Web application would perform no further authentication or cross-checking of form submissions, and the order would be dispatched at a discounted price!

```
<FORM METHOD="LINK" ACTION="/shop/checkout.htm">
<INPUT TYPE="HIDDEN" name="quoteprice"
value="4.25">Quantity: <INPUT TYPE="text"
NAME="totalnum"> <INPUT TYPE="submit" VALUE="Checkout">
</FORM>
```

This practice is still common on many sites, though to a very lesser degree.

# Loose Authentication Mechanism

---

- An example of this is a Web application providing access to various documents as follows. The application requires authentication and then presents a menu of documents the user is authorized to access, like this:

<http://www.server.com/showdoc.asp?docid=10>

- Although reaching the menu requires authentication, the showdoc.asp script requires no authentication itself and blindly provides the requested document.

---

THANK YOU !!